

Bit Manipulation

Bitwise Operator —

AND (&)

OR (|)

INVERSE (~)

XOR (^)

LEFT SHIFT (<<)

RIGHT SHIFT (>>)

TRUTH TABLE

a	b	a & b	a b	a ^ b	~a
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

$\wedge \rightarrow$ addition without carry.

$$0 + 0 \rightarrow 0$$

$$1 + 1 \rightarrow$$

2/2 $\rightarrow$ it not exist in xor

$$\begin{array}{r} 1 \\ + 1 \\ \hline 2 \text{ i.e. } 2 \\ \uparrow 0 \end{array}$$

* Properties of Binary Operators —

$$14 \rightarrow 1110$$

$$19 \rightarrow 10011$$

$$12 \rightarrow 1100$$

$$(14 \& 12) \leftrightarrow (1110) \& (1100)$$

$$\begin{array}{c}
 1100 \\
 (14 \mid 12) \longleftrightarrow (1110) \mid (1100) \\
 1110
 \end{array}$$

$$\begin{array}{r}
 10011 \\
 \wedge 10110 \\
 \hline
 00101
 \end{array}$$

$$\begin{array}{l}
 (14 \& 1) \longrightarrow 1110 \& 0001 \\
 \Rightarrow 0000
 \end{array}$$

$$\begin{array}{l}
 (12 \& 1) \longrightarrow 1100 \& 0001 \\
 \Rightarrow 0000
 \end{array}$$

$$\begin{array}{l}
 19 \& 1 \longrightarrow 10011 \& 00001 \\
 \rightarrow 00001
 \end{array}$$

$$\begin{array}{l}
 3 \& 1 \longrightarrow 11 \& 01 \\
 \Rightarrow 01
 \end{array}$$

Observations: -

(i) If number is odd $\& 1 = 1$

(ii) If number is even $\& 1 = 0$

hence -

$$(i) \left[a \& 1 = \begin{cases} 0 & a \text{ is even} \\ 1 & a \text{ is odd} \end{cases} \right]$$

$$\frac{1}{1} \quad \frac{1}{1} \quad \frac{1}{1} \quad \frac{1}{1}$$

$$(ii) \quad [a \vee a = a] \Leftrightarrow \begin{array}{cccc} 1 & 1 & 0 & 1 \\ \vee & 1 & 1 & 0 & 1 \\ \hline 1 & 1 & 0 & 1 \end{array}$$

$$(iii) \quad [a \wedge a = a] \Leftrightarrow \begin{array}{cccc} 1 & 1 & 0 & 1 \\ \wedge & 1 & 1 & 0 & 1 \\ \hline 1 & 1 & 0 & 1 \end{array}$$

$$(iv) \quad [a \vee 0 = a] \wedge \begin{array}{cccc} 1 & 1 & 0 & 1 \\ \vee & 1 & 1 & 0 & 1 \\ \hline 0 & 0 & 0 & 0 \end{array}$$

$$(v) \quad [a \vee 0 = 0] \Leftrightarrow \begin{array}{cccc} 1 & 1 & 0 & 1 \\ \vee & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 \end{array}$$

$$(vi) \quad [a \wedge 0 = a] \Leftrightarrow \begin{array}{cccc} 1 & 1 & 0 & 1 \\ \wedge & 0 & 0 & 0 & 0 \\ \hline 1 & 1 & 0 & 1 \end{array}$$

$$(vii) \quad [a \wedge 0 = a] \wedge \begin{array}{cccc} 1 & 1 & 0 & 1 \\ \wedge & 0 & 0 & 0 & 0 \\ \hline 1 & 1 & 0 & 1 \end{array}$$

Note $\rightarrow$ Bitwise operators are commutative in nature - which means -

$$a \& b = b \& a$$

$$a | b = b | a$$

$$a \wedge b = b \wedge a$$

$$a \& b \& c = c \& b \& a$$

Note $\rightarrow$ All bitwise operators can be perform in any order, output remain same.

ex - $a \wedge b \wedge c \wedge b \wedge a \wedge c = ?$

Solⁿ we can write it like below also

$$a \wedge a \wedge b \wedge b \wedge c \wedge c$$

$$\Rightarrow 0 \wedge 0 \wedge 0$$

$$\Rightarrow 0 \text{ Ans}$$

(ii) $a \wedge b \wedge c \wedge b \wedge c$

$$\Rightarrow a \wedge b \wedge b \wedge c \wedge c$$

$$\Rightarrow a \wedge 0 \wedge 0$$

$$\Rightarrow a \text{ Ans}$$

$$\begin{array}{r} 01 \\ \wedge 10 \\ \hline 11 \end{array} \quad \begin{array}{l} 11 \times 2^0 = 1 \\ 1 \times 2^1 = 2 \end{array}$$

Q. Given N numbers where each number is occurring 2 times except one. Find that number.

$$\begin{array}{r} 11 \\ 10 \\ \hline \end{array}$$

0 1

0 1

$a = [1, 1, 2, 2, 3, 3, 5]$

Try Brute force

Optimize

```

longsamne-
xor = 0
for i in n {
    xor = xor ^ i
}
return xor

TC = O(n)
SC = O(1)
    
```

10
11
11

xor xor
0 ^ 1 1
1 ^

1, 2, 2, 3, 1

xor = xor ^ 1
1 = 0 ^ 1
3 = 1 ^ 2
= 3 ^ 2
1 = 1 ^ 3
2 = 2 ^ 1
③ ← ans

Left shift <<

11
10

$a = 10$ (8 bit number)

$a =$

7	6	5	4	3	2	1	0
0	0	0	0	1	0	1	0

$10 = 10 \times 2^0$

$a << 1$

7	6	5	4	3	2	1	0
0	0	0	1	0	1	0	0

$20 = 10 \times 2^1$

$a << 2$

7	6	5	4	3	2	1	0
0	0	1	0	1	0	0	0

$40 = 10 \times 2^2$

7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1

... 3

$a \ll 5$

0	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

$$80 = 10 \times 2$$

$a \ll 4$

1	0	1	0	0	0	0	0
---	---	---	---	---	---	---	---

$$160 = 10 \times 2^4$$

$a \ll 5$

0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

64

↑
↓
Overflow

Now let suppose No overflow (Sufficiently large DT)

$$a \ll 1 \Rightarrow a \times 2^1$$

$$a \ll 2 \Rightarrow a \times 2^2$$

$$a \ll 3 \Rightarrow a \times 2^3$$

⋮

$$a \ll n \Rightarrow a \times 2^n$$

Q.1 Given two binary strings A and B.

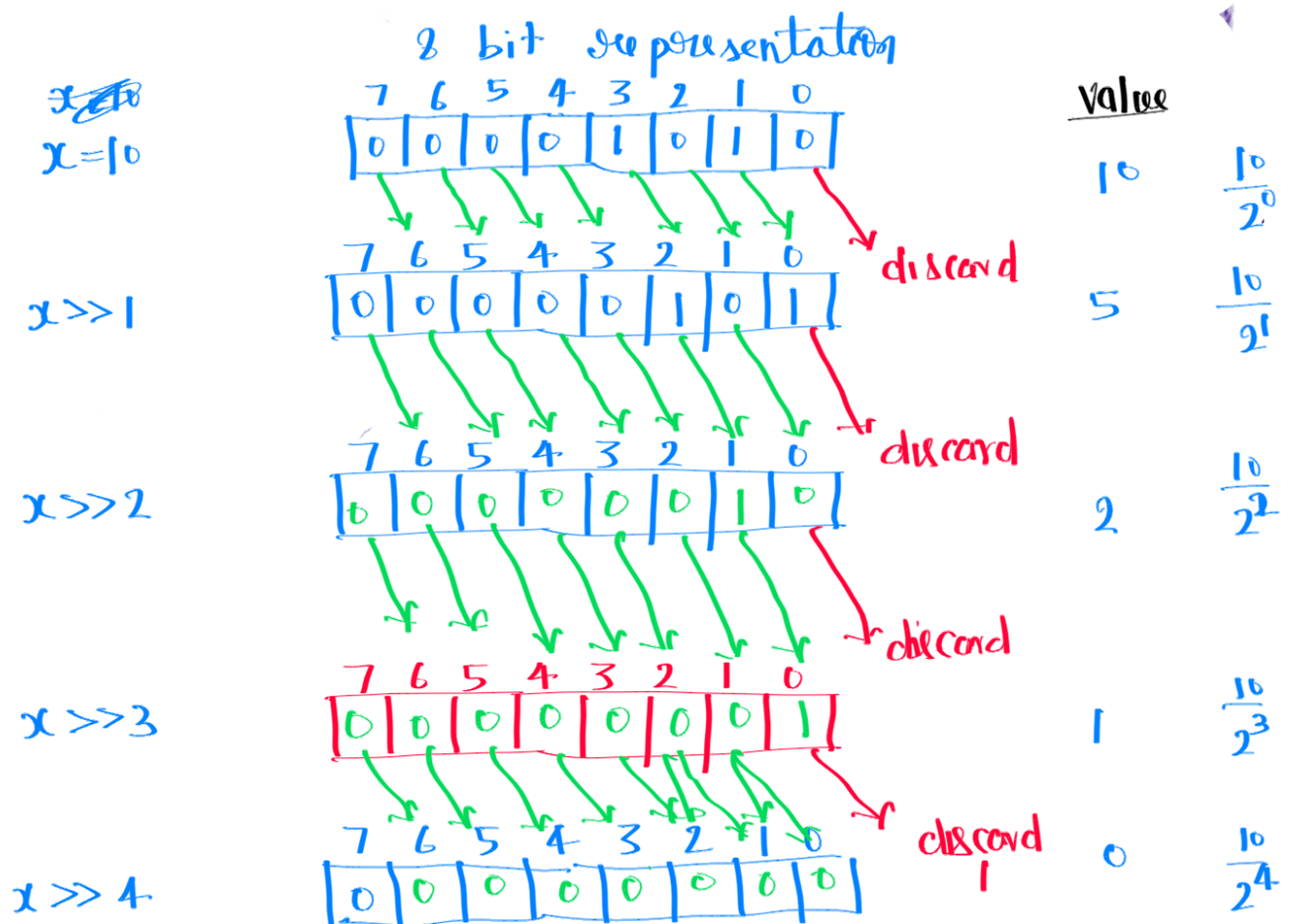
Return their sum.

Q.2. WAP that takes an integer

and returns the number of 1 bits present in its binary representation

Q.3 Reverse the bits of an 32 bit unsigned integer A.

Right Shift :—



Nik →
G)

Left shift multiply by 2
 $\therefore x \ll i \Leftrightarrow x \times 2^i$

Right shift divide by 2

$$\therefore x \gg d \leftrightarrow \frac{x}{2^d}$$

(ii) Left shift can cause overflow

(obviously since numbers are being multiply by 2 and at one point it will go out of boundaries (8bit exp))

$$3 \xrightarrow{LSI} 6 \xrightarrow{LSI} 12 \xrightarrow{LSI} 24$$

Right shift can not cause overflow.

Q. Given N, i check if ith bit of N is set or unset.

Solⁿ

$$N = 21$$

7	6	5	4	3	2	1	0
0	0	0	1	0	1	0	1

$$i = 0 \rightarrow \text{set}$$

$$i = 3 \rightarrow \text{unset}$$

Bruteforce - ~~Try IT.~~

- convert to Binary
- check the ith Bit

Optimize -

let suppose we have $N = 10101$ then
how we can say that selected bit is
set or not?

if we take $\&$ with 1 then we
can find out this $\rightarrow$ how?

$$\begin{array}{r} 10101 \\ \& 00001 \\ \hline 1 \end{array} \qquad \begin{array}{r} 10000 \\ \& 00001 \\ \hline 0 \end{array}$$

0th bit set or not

if $(N \& 1) == 1$ return true

1th bit set or not?

now, $N = 1010$ 

$$\begin{array}{r} N \gg 1 \\ 01010 \\ 00001 \\ \hline 0 \end{array}$$

so for $(N \gg 1) \& 1 == 1$ return true

2nd bit?

$$(N \gg 2) \& 1 == 1$$

Programme -

```
func checkBit(N, i) → Bool {  
    return (N << i) & 1  
}
```

Q. Count number of set bits in a number

Soln

1 0 1 0 1

Programme-1

countSetBits(- n : Int) → Int

count = 0

for i in 0..32 {

if ((n << i) & 1)

count += 1

}

return count

Since n is an Int

Programme-2

func countSetBits(N) → Int {

var count = 0

```

while N > 0 {
    if N & 1 {
        count += 1
    }
    N = N >> 1
}
return count
}

```

$T.C = O(\log n)$

Q. Given x, y set the x^{th} bit & y^{th} bit of zero (0).

$y = 3$ $x = 5$

5	4	3	2	1	0
1	0	1	0	0	0

$2^5 + 2^3 \Rightarrow 40$

$$\begin{aligned}
 & 1 \times 2^4 + 1 \times 2^3 \\
 & 1 \times 2^5 \\
 \Rightarrow & 100000 + 1000 \\
 = &
 \end{aligned}$$

<pre> if x == y return 2^x else return 2^x + 2^y </pre>	<pre> if x == y return (1 << x) else return (1 << x) + (1 << y) </pre>
---	--

↓

```
return [(1 << x) | (1 << y)]
```

Q. Given a number n , set i^{th} bit of N

$$N = 21 \\ = 10101$$

$$\text{return } N | (1 \ll i)$$

Q. Unset i^{th} bit of a number

1 0 1 0 1 1

↓

1 0 1 0 0 1

$$(n \gg i) \& 0$$

← incorrect

$$(N \& [\sim (1 \ll i)])$$

Note → unset a bit : AND → 1110111
set a bit : OR → 0001000

Q. Set bits in range $[i, j]$

input -

8	7	6	5	4	3	2	1	0
	0	0	0		0	0		0

o/p - 1 0 0 1 1 1 0 1 0